

Session 1 (from code to container)

what we cover today :-

1. MLOps Maturity Model
2. Python Packaging
3. FastAPI
4. Serialization Formats
5. Docker & Containerization
6. Structured Logging
7. Unit Testing With Pytest

1- MLOps Maturity Model

We have 3 questions when we join a new team ?

1. can i reproduce any past experiment from the artifact store (like an old data i trained the model on it can i go back to it or that is impossible)
2. how long does model go from training to production (if i have a new data i need to pause the model to train again? and how long does this operation take? Is there any automatic pipeline automating this operation?)
3. Who do I speak to when the model starts performing poorly in production (Like data drifting) ?

We have 5 levels of MLOps :-

1. **No MLOps:-**
it is like notebook every thing is manually .
2. **Manual Process :-**
convert from notebooks to scripts but still deploying testing..etc is manually.
3. **ML pipeline:-**
Automated training & experiment tracking and basic CI .
4. **CD for ML :-**
Auto retraining on trigger (continue integrated and continues Development) .
5. **Full MLOps :-**
Zero Touch pipeline all done automatically (detect new data and train , develop , test , deploy and integrated it into the system) .

```

notebook_final_v3_FINAL_USE_THIS.ipynb

1 import pandas as pd
2 import pickle

4 df = pd.read_csv('/Users/ahmed/Desktop/data_final.csv')

6 # trained last week, don't retrain
7 model = pickle.load(open('model_v2_good.pkl', 'rb'))

9 # this works on my machine
10 pred = model.predict(df[features])

```

Hardcoded path

Breaks on any other machine.

No input validation

Crashes on unexpected data.

No API

Nobody else can call it.

No version

Which model is in production right now?

what are the problems here ?

1. **the paths are hardcoded** (the code will run on my pc only because the files and the file paths is on my pc it will break on other pcs).
2. **there isn't input validation** (it will crash on unexpected data) .
3. **No APIS** (nobody else can call it outside this code) .
4. **No Version** (which version on production ? (model version and data version)).

note :- search on **VAULT** .

2- Python project structure

1. **src/** : it is an importable folder (because of `__init__`) it has model implementation and preprocessing .
2. **tests/** : it has the test files (pytest search on it) CI ready .
3. **configs/** : project configs (ENV Variables) .
4. **notebooks/** : all notebooks related for this project.
5. **Dockerfile** : reproducible runtime - same ENV every where (containrize the project).
6. **pyproject.toml** : all tools and dependencies and metadata used in the project with its version (Like requirements.txt) .
7. **README.md** : this is a readable file to explain the project and how to run it.

- **the power of uv :-**
 - **uv** is the replacement for **pip**
 - **uv is faster than pip from 10 to 100 times**
 - **when you begin a project -> uv init project-name**
 - it create folder have (README.md file , pyproject.toml , main.py)
 - when adding any module using uv it automatically write it inside the pyproject.toml (dependencies section).
 - uv lock :- has all versions you used in your project pushed it on github to make any one in your team to work on the same versions.
- **OOP is important!!!!!!!!!!!!!!!!!!!!!!**
- **search on Abstraction and Composition ????????????????**

3- Framework (FASTAPI)

- **with frame work :-**
 - you can validate the input using pydantic module.
 - http and https handling.
 - generate docs - interactive ui to know the api i have .
 - support Async (process all users at the same time).
- **without framework :-**
 - No input validation.
 - No http handling (you should build the socket from scratch).
 - no docs .
 - no async - one user block all until finish processing.

TOPIC 03 · FASTAPI



Your model as a REST API

About twenty lines

```
src/api.py
1 from fastapi import FastAPI
2 from pydantic import BaseModel, Field
3 from src.model import RideDurationModel
4
5 app = FastAPI(title="Ride Duration API")
6 model = RideDurationModel() # loaded once
7
8 class PredictRequest(BaseModel):
9     distance_km: float = Field(..., gt=0)
10     passengers: int = Field(1, ge=1, le=8)
11
12 class PredictResponse(BaseModel):
13     duration_min: float
14
15 @app.post("/predict", response_model=PredictResponse)
16 async def predict(req: PredictRequest):
17     d = model.predict([req.distance_km, req.passengers])
18     return PredictResponse(duration_min=round(d, 2))
```

Pydantic validation

Field(gt=0) rejects negative distances. 422 before predict() runs.

Load the model once

At module level. Inside predict() it reloads on every request.

response_model

FastAPI validates your response too — it fails loudly in dev.

Run it

uvicorn src:api:app --reload → localhost:8000/docs

1. **Load the model once** : at the beginning of the script load the model once and use it inside the script because if you load it inside predict it will load every request and that is not good.
2. **pydantic validation** :
 1. import BaseModel and validate the request and the response datatype
 2. and import Field to validate the value of the variable (default , grater or equal (ge) , gtater than (gt) , less or equal (le) , less than (lt)).
 3. custom validator :


```
@field_validator("field_you_want_to_validate")
@classmethod
def Validation_function():
    your custom validation function
    if true → all is ok status_code 200 OK
    if not true → there is an error status_code 422
```
3. **Run it** : uvicorn src.api:app --reload-> localhost:8000/docs.

- we also have litestar (api framwork (sresearch!!!!!!!!!!!!!!!!!!!!!!))
- we have get api `@app.get("/health")` because in a big project kubernetes search on /health api to ensure that the api work well!!!!!!!!!!

TOPIC 03 · CHECKLIST



What every ML endpoint must have

01	Input validation	Reject malformed data before it reaches the model	Pydantic BaseModel with Field constraints
02	Health endpoint	Load balancers need /health to route traffic	<code>@app.get("/health")</code> returning <code>{status: healthy}</code>
03	Model loaded once	Loading on every request adds 200–500ms	Instantiate at module level, reuse across requests
04	Async handlers	Sync handlers block the server under load	<code>async def predict()</code>
05	Response schema	Unvalidated responses hide bugs until production	<code>response_model=PredictResponse</code>
06	Auto-generated docs	Your API is useless if nobody can call it	<code>/docs</code> and <code>/redoc</code> , zero code

- it is a checklist to check that you build good APIs

4- Serialization formats :-

TOPIC 04 · SERIALIZATION



Serialization formats

FORMAT	TYPE	SPEED	SIZE	HUMAN READABLE	BEST FOR
JSON	Text	Slow	Large	Yes	REST APIs, configs, logging
MessagePack	Binary	Fast	Small	No	High-throughput inter-service calls
Protobuf	Binary	Fast	Tiny	No	gRPC services, streaming pipelines
Pickle	Binary	Fast	Medium	No	Python-only model files — never in APIs
ONNX	Binary	—	Medium	No	Cross-framework model portability

Never serialize models with Pickle and expose them via an API. Pickle is not safe to deserialize from untrusted sources.

20 / 46

1. **JSON** : it is a **readable** format, use it in **request and response** body .
2. **MessagePack** : faster than **JSON** but **unreadable**, use it to **send messages between systems**.
3. **Protobuf** : smallest one , use it in **data streaming**.
4. **Pickle** : binary file work with **python file** , **note that !!! never use pickle in APIs** because it is not safe if used in an untrusted source!!!!!!!!!!!!!!
5. **ONNX** : **the most usage in real work**
 1. train in any module you want (**pttorch** , **tensorflow** , **sklearn ..etc**) on (**nootbooks** , **python scripts** , **..etc**) and serve it with **ONNX Runtime** in production - no **PyTorch** in production (because it is heavy)
 2. Export your file to the .onnx graph
 3. optimize your system

4. Run any where (edge , mobile , gpu , java , C++ , ...etc)

- the power of ONNX :-

1. **FrameWork independence** : train in PyTorch (as example) , serve it with ONNX Runtime .
2. **Cross-Language Serving** : Run the same file from C++ , java , ...etc.
3. **Smaller Production image** : ONNX runtime is about **15 MB**
4. **Hardware acceleration** : execution on all providers target **CUDA , CPU , OpenVINO , ...etc.**

- how to export a pytorch model into ONNX

TOPIC 04 · ONNX EXPORT



Exporting a PyTorch model

From trained weights to a portable graph

```
export_onnx.py

1 import torch, onnx
2
3 model = RideDurationTorchModel()
4 model.eval()          # disable dropout / batchnorm
5
6 dummy = torch.randn(1, 2) # shape only
7
8 torch.onnx.export(
9     model, dummy, "model.onnx",
10    export_params=True,
11    opset_version=17,
12    input_names=["features"],
13    output_names=["duration"],
14    dynamic_axes={"features": {0: "batch"},
15                  "duration": {0: "batch"}},
16 )
17
18 onnx.checker.check_model(onnx.load("model.onnx"))
```

model.eval()

Forgetting this leaves dropout in training mode — silently wrong predictions.

dummy_input

Only shape and dtype matter. PyTorch traces the graph by running it once.

dynamic_axes

Without it the model is locked to batch size 1.

onnx.checker

Validate before shipping. A broken export fails silently in some runtimes.

?? / 46

1. **model.eval()** : forget the leaves dropout in training mode
2. **dummy_input** : shape the input only
3. **export_params** : export the model parameters (weights and bias).
4. **dynamic_axes** : make the model to process many input and many output (num of batches) without it the model will serve one batch in the input and the output.
5. **onnx.checker** : validate before shipping



What converts, and how well

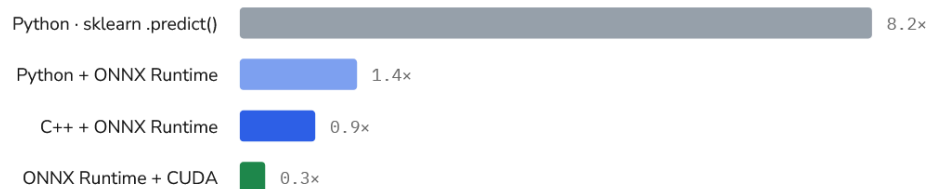
SOURCE FRAMEWORK	CONVERTER	NOTES
PyTorch	torch.onnx (built in)	Best supported — native export API
scikit-learn	skl2onnx	Covers most estimators and Pipelines
XGBoost / LightGBM	onnxmltools	Tree ensembles convert well
TensorFlow / Keras	tf2onnx	Some custom layers need manual ops
Hugging Face	optimum.onnxruntime	Transformers export with attention support

24 / 46



What you actually gain

Same model, same inputs — only the runtime changes



Relative inference latency. The ratio is the point, not the absolute milliseconds.

25 / 46



When to use it, and when not to

USE ONNX WHEN

- You serve from C++, Java, C# or the browser
- You want to drop a training framework from your image
- You target edge devices — mobile, Raspberry Pi, embedded
- You need TensorRT, OpenVINO or CoreML acceleration
- Multiple teams need the model without your Python env

SKIP ONNX WHEN

- You are serving from Python anyway
- Your model uses custom ops the converter cannot handle
- You need to keep fine-tuning the same artifact
- The model changes weekly — one more CI step to maintain
- Your current latency is already well within SLA

- **use ONNX on :-**
 - a. you serve from C++ , Java ,...etc
 - b. you want to drop training from your image .. train on any module you want and convert it to ONNX to serve with others
 - c. you target to upload model on edge (mobile , Raspberry Pi , ...etc).
 - d. you need hardware acceleration (TensorRT , OpenVINO , ...etc) .
 - e. multiple teams need the model without your Python env.
- **Skip ONNX in :-**
 - a. you serve from Python anyway (you don't need anything except python).
 - b. your model needs custom operations that can't be converted to ONNX so you need to skip ONNX to work well .
 - c. you are in fine tuning experimentation .
 - d. the model change weekly and you havn't CI to maintain this point .
 - e. your latency is already well without ONNX .
- **all big companies (need fast operations) use C++ , C , CUDA , Rust ...etc in production !!!!!!!!!!!!!!!**

5- Docker (ship the environment not just the code)

- **the problem**
 - it work on my machine (that is the problem)
 - EX : python 3.9 locally VS python 3.11 on server
 - different os path
- **the Docker solution :**
 - frozen versions and packages
 - same image from dev→CI→staging→production
 - Run any where Docker is installed
- **Docker Component**
 - **Image** : it is the base of container (Like oop class)
 - **container** : it is the running instance (Like an object)
 - **Dockerfile** : it is the file that build the image layer by layer
 - **Registry** : Docker hub it is like GitHub but for Docker Images
- **Dockerfile :**
 - it is a text file use to create a docker img and run container based on my img with my dependencies .

TOPIC 05 · DOCKERFILE



Dockerfile anatomy

Every line is a frozen decision

```
Dockerfile
1 # Stage 1: build
2 FROM python:3.11-slim AS builder
3 WORKDIR /app
4 COPY pyproject.toml .
5 RUN pip install --no-cache-dir -e .[dev]
6
7 # Stage 2: runtime
8 FROM python:3.11-slim AS runtime
9 WORKDIR /app
10 COPY --from=builder /usr/local/lib/python3.11 \
11 /usr/local/lib/python3.11
12 COPY src/ ./src/
13
14 RUN adduser --disabled-password appuser
15 USER appuser
16
17 EXPOSE 8000
18 CMD ["uvicorn", "src.api:app", "--host", "0.0.0.0"]
```

Multi-stage build

Builder installs everything, runtime copies only what runs. 60% smaller.

Layer caching

COPY pyproject.toml before COPY src/. Deps change rarely, code changes often.

Non-root user

Never run as root. If the app is exploited, damage is contained.

CMD vs ENTRYPOINT

CMD is the default command. ENTRYPOINT is the fixed binary.

- **we should COPY dependencies before COPY src/**
 - a. to load dependencies before source code
 - b. to benefit from cached layers (if build image again it won't download the dependencies that downloaded before).
- **Docker-compose :**
 - a. it is a dockerfile can run more than 1 service together with a constant volume

6- LOGING & testing:

- it is like print but it is more professional we can print (user name - timedate - the error or info or warning message .. etc).
- **unit testing with pytest:**
 - it is a library that helps you to unit and functional testing (Search!!!!!!)